CSC1015F Assignment 4

Control (if, for, while)

Learning objectives

By the end of this assignment, you should be able to:

- Understand the need for iteration (otherwise known as looping) and the difference between the two major forms:
  - counter-controlled (for) and condition-controlled (while) loops.
- Be able to identify when and where to use the break, continue, else, and pass statements, which can be used to modify the behaviour of a loop structure.
- Be able to use for and while loops effectively in your programs.

Assignment Instructions

This assignment involves constructing Python programs that use input and output statements, '`if`' and '`if-else`' control flow statements, '`for`' statements, '`while`' statements and statements that perform numerical manipulation.

NOTE Your solutions to this assignment will be evaluated for correctness. Furthermore, your solutions will also be evaluated for the following qualities:

- Documentation
  - Use of comments at the top of your code to identify program purpose, author and date.
  - Use of comments within your code to explain each non-obvious functional unit of code.
- General style/readability
  - The use of meaningful names for variables and functions.
- Algorithmic qualities
  - Efficiency, simplicity

These criteria will be manually assessed by a tutor and commented upon. In this assignment, up to 10 marks will be deducted for deficiencies.

Question 1 [30 marks]

Many integers become palindromes if you repeatedly "reverse the digits and add".

For example, 87 → 87+78=165 → 165+561=726 → 726+627=1353+3531 → 4884 (palindrome).

Task:

Write a program called `revadd_palindrome.py` that:

1. Accepts a **non-negative integer** n.
2. Repeatedly performs `n = n + reverse(n)` until n is a palindrome **or** a maximum of **25 iterations** is reached.
3. Prints the full sequence on one line (space-separated), ending with the palindrome if one occurs within 25 steps.
4. If no palindrome appears within 25 iterations, print out the sequence generated and the message "`No palindrome found in 25 iterations.`" (see the last *Sample IO below*).
5. Input validation:
    - Non-integers → "`Invalid input. Please enter a valid integer.`"
    - Negative integers → "`Please enter a non-negative integer.`"
6. **DO NOT use lists.**
7. ***DO NOT*** use string slicing.

Hints:

- Use a **while loop** with a counter (condition-controlled).
- Demonstrate break when a palindrome is reached.
- reverse the number using **pure arithmetic (modulus and integer division)**.

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a non-negative integer:
87
87 165 726 1353 4884
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a non-negative integer:
-78
Please enter a non-negative integer.
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a non-negative integer:
q
Invalid input. Please enter a valid integer.
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a non-negative integer:
196
196 887 1675 7436 13783 52514 94039 187088 1067869 10755470 18211171
35322452 60744805 111589511 227574622 454050344 897100798 1794102596
```

```
8746117567 16403234045 70446464506 130992928913 450822227944
900544455998 1800098901007 8801197801088
No palindrome found in 25 iterations.
```

Question 2 [30 marks]

Write a program called `digitproduct.py` that:

1. Accepts a *positive integer* from the user.
2. Repeatedly computes the *product of its digits*.
3. Continues until the result is *a single-digit number*.
4. Prints the *entire sequence on one line separated by spaces*.
5. **DO NOT use lists**.

For example:

Starting with $n = 39$,

Step	Calculation	Result
Start	–	39
1	3×9	27
2	2×7	14
3	1×4	4

Sequence produced:

```
39 27 14 4
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a positive integer:
39
39 produces the sequence:
39 27 14 4
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a positive integer:
97
97 produces the sequence:
97 63 18 8
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a positive integer:
-87
Please enter a positive integer.
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter a positive integer:
q
Invalid input. Please enter a valid integer.
```

Question 3 [40 marks]

Write a program called `rectanglegrid.py` that accepts a starting number, `n`, and two additional inputs: the number of rows (`r`) and columns (`c`). The program should print a grid of numbers, starting from `n`, with `r` rows and `c` columns. **DO NOT use lists.**

Output will take the following form:

```
[d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d
[d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d
[d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d
[d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d
[d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d
[d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d [d][d]d
```

That is, numbers are printed using a field width of 3 and are right justified. ('`[d]`' represents an optional digit.) Fields are separated by a single space. There are no spaces after the final field.

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter the starting number (n):
2
Enter the number of rows (r):
6
Enter the number of columns (c):
9
  2   3   4   5   6   7   8   9  10
 11  12  13  14  15  16  17  18  19
 20  21  22  23  24  25  26  27  28
 29  30  31  32  33  34  35  36  37
 38  39  40  41  42  43  44  45  46
 47  48  49  50  51  52  53  54  55
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter the starting number (n):
88
Enter the number of rows (r):
6
Enter the number of columns (c):
8
 88  89  90  91  92  93  94  95
 96  97  98  99 100 101 102 103
104 105 106 107 108 109 110 111
112 113 114 115 116 117 118 119
120 121 122 123 124 125 126 127
128 129 130 131 132 133 134 135
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter the starting number (n):
78
Enter the number of rows (r):
-8
Enter the number of columns (c):
5
Rows and columns must be positive integers.
Enter the starting number (n):
56
Enter the number of rows (r):
6
Enter the number of columns (c):
-4
Rows and columns must be positive integers.
Enter the starting number (n):
78
Enter the number of rows (r):
5
Enter the number of columns (c):
8
 78  79  80  81  82  83  84  85
 86  87  88  89  90  91  92  93
 94  95  96  97  98  99 100 101
102 103 104 105 106 107 108 109
110 111 112 113 114 115 116 117
```

Sample IO *(The input from the user is shown in* ***bold*** *font – do not program this*):

```
Enter the starting number (n):
q
Invalid input. Please enter integers.
Enter the starting number (n):
2
Enter the number of rows (r):
w
Invalid input. Please enter integers.
Enter the starting number (n):
4
Enter the number of rows (r):
4
Enter the number of columns (c):
3
  4   5   6
  7   8   9
 10  11  12
 13  14  15
```

HINT: Use a '`for`' loop within a '`for`' loop for rows and columns. Use a while loop to accept input.

Submission

Create and submit a Zip file called '`ABCXYZ123.zip`' (where ABCXYZ123 is YOUR student number) containing `revadd_palindrome.py`, `digitproduct.py`, and `rectanglegrid.py`.

NOTES:

1. FOLDERS ARE NOT ALLOWED IN THE ZIP FILE.
2. As you will submit your assignment to the Automarker, the Assignment tab may say something like "Not Complete". THIS IS COMPLETELY NORMAL. IGNORE IT.